

CAPGEMINI 2027 – SECTION 2 DEBUGGING ASSESSMENT

Engineering Students – Practice Set
Difficulty: Mid to Advanced

Instructions

For each question: (1) Read the problem and code carefully. (2) Locate the syntax, logical, runtime, or design issue. (3) Select the best correction. (4) Test normal cases and edge cases.

Q1. C – String Parsing with Boundaries

The function should return the first character that appears exactly twice in a string. If no such character exists, return '\0'. Identify and correct the logical issue in the code.

```
char firstRepeated(string s) {  
    unordered_map<char,int> freq;  
    for(char ch : s) freq[ch]++;  
    for(char ch : s)  
        if(freq[ch] == 2)  
            return ch;  
    return '\0';  
}
```

- A. Replace `freq[ch] == 2` with `freq[ch] > 1`
- B. The function is correct for the stated requirement
- C. Iterate over the map instead of the string
- D. Sort the string before counting

Q2. C – Array Rotation

The function should rotate an array to the right by `k` positions. For `k` greater than the array length, it should still work correctly. Identify the issue.

```
void rotate(vector<int>& a, int k) {  
    k = k % a.size();  
    reverse(a.begin(), a.end());  
    reverse(a.begin(), a.begin()+k);  
    reverse(a.begin()+k, a.end());  
}
```

- A. `k` must be reduced using `a.size()-1`
- B. The reversal sequence is invalid for right rotation
- C. The array must be sorted first
- D. `reverse` cannot be used with vectors

Q3. C++ – Binary Search First Occurrence

The function should return the first index of key in a sorted array containing duplicates. It currently returns an arbitrary matching index. What change is required?

```
int firstPos(vector<int>& a, int key) {  
    int l=0, r=a.size()-1, ans=-1;  
    while(l<=r) {  
        int mid=l+(r-l)/2;  
        if(a[mid]==key) {  
            ans=mid;  
            l=mid+1;  
        } else if(a[mid]<key) l=mid+1;  
        else r=mid-1;  
    }  
    return ans;  
}
```

- A. Change `l = mid + 1` to `r = mid - 1` after a match
- B. Change `r = mid - 1` to `l = mid`
- C. Return `mid` immediately
- D. Start `l` at 1

Q4. C++ – Pointer Lifetime

The function returns a pointer to the maximum element. Identify the issue.

```
int* maxElement(vector<int> a) {  
    int* p = &a[0];  
    for(int i=1; i<a.size(); i++)  
        if(a[i] > *p) p=&a[i];  
    return p;  
}
```

```
return p;
}
```

- A. The pointer must be initialized to NULL
- B. The vector is passed by value, so the returned pointer refers to destroyed storage
- C. p must be a double pointer
- D. The comparison must use >= only

Q5. C++ – Linked List Deletion

The function deletes the first node containing key. It crashes when the head itself contains key. Identify the missing case.

```
void deleteKey(Node*& head, int key) {
    Node* cur=head;
    while(cur && cur->next) {
        if(cur->next->data==key) {
            Node* t=cur->next;
            cur->next=t->next;
            delete t;
            return;
        }
        cur=cur->next;
    }
}
```

- A. Handle deletion when head is NULL only
- B. Handle deletion of head before traversing from head->next
- C. Move cur two nodes at a time
- D. Delete cur instead of cur->next

Q6. C++ – Stack Underflow

The function evaluates a postfix expression. It sometimes crashes on malformed input. Which validation is required before applying an operator?

```
int apply(stack<int>& st, char op) {
    int b=st.top(); st.pop();
    int a=st.top(); st.pop();
    return a+b;
}
```

- A. Check that at least two operands exist before popping
- B. Check that the stack is sorted
- C. Push zero before every operator
- D. Replace stack with queue

Q7. C++ – Tree Diameter

A binary-tree diameter implementation returns the longest path measured in edges, but the code returns a value one greater than expected. Identify the likely issue.

```
int diameter(Node* root) {
    if(!root) return 0;
    return max(height(root->left)+height(root->right)+1,
        max(diameter(root->left), diameter(root->right)));
}
```

- A. The diameter formula counts nodes while the expected measure counts edges
- B. height must always return -1
- C. The left subtree must be ignored
- D. diameter cannot be recursive

Q8. C++ – BST Validation

The following validation incorrectly accepts some invalid BSTs. What is the robust correction?

```
bool valid(Node* root, int low, int high) {
    if(!root) return true;
    if(root->data <= low || root->data >= high) return false;
    return valid(root->left, low, root->data) &&
        valid(root->right, root->data, high);
}
```

- A. Use a wider numeric type or optional bounds to avoid sentinel collisions
- B. Replace && with ||
- C. Remove all bounds
- D. Compare only each node with its parent

Q9. C++ – BFS Distance

A BFS shortest-distance function uses a queue but assigns distance when a node is removed. It may enqueue the same vertex many times. What is the best correction?

```
for(int v : adj[u]) {  
    if(dist[v] == -1) {  
        q.push(v);  
    }  
}
```

- A. Mark dist[v] when enqueueing, before pushing it
- B. Mark dist[v] only after the entire BFS
- C. Set every distance to zero
- D. Use DFS instead

Q10. C++ – DFS Backtracking

A recursive path-search function leaves visited nodes marked after returning, causing valid paths to be skipped. What should be considered?

```
bool path(Node* u) {  
    visited[u]=true;  
    for(Node* v : adj[u])  
        if(!visited[v] && path(v)) return true;  
    return false;  
}
```

- A. Always unmark visited on return, regardless of graph type
- B. Distinguish global graph visitation from path-local backtracking state
- C. Remove the visited array
- D. Mark only the destination

Q11. C++ – Grid DP Obstacles

A robot must count paths through a grid containing blocked cells marked -1. The implementation adds paths from blocked cells. What is the required rule?

```
if(grid[i][j] == -1)
    dp[i][j] = 1;
else
    dp[i][j] = dp[i-1][j] + dp[i][j-1];
```

- A. Set blocked-cell paths to 0 and do not use them in transitions
- B. Set blocked-cell paths to 1
- C. Skip only the first row
- D. Replace addition with multiplication

Q12. C++ – LCS Reconstruction

The LCS length is correct, but reconstruction produces a reversed string. What should be done after tracing from `dp[m][n]`?

- A. Reverse the reconstructed sequence before returning it
- B. Sort the reconstructed sequence
- C. Add every unmatched character
- D. Use the DP table's first row only

Q13. Java – Mutable String Keys

A developer stores objects as keys in a `HashMap`. After inserting a key, a field used by `hashCode()` is changed. Lookups may fail. Why?

- A. `HashMap` keys must remain stable with respect to `equals()` and `hashCode()` while stored
- B. `HashMap` supports only `String` keys
- C. `hashCode()` must return a random value
- D. Changing a key always updates its bucket automatically

Q14. Java – Integer Comparison

A program compares two `Integer` objects using `==` and gets inconsistent results for larger values. What is the correct fix?

- A. Use `==` because `Integer` objects are primitive
- B. Use `equals()` for value comparison or unbox safely
- C. Convert both values to `String`
- D. Compare their hash codes

Q15. Java – Exception Safety

A method opens a file and may throw an exception before `close()` is reached. Which construct is most appropriate?

- A. `try-with-resources`
- B. A larger `finally` block without closing
- C. Ignore the exception
- D. Open the file repeatedly

Q16. Java – Concurrent Modification

A program removes elements from an `ArrayList` inside an enhanced for-loop and throws `ConcurrentModificationException`. Which solution is generally valid?

- A. Use `Iterator.remove()` or collect items for removal after iteration
- B. Replace `remove` with `get`
- C. Catch the exception and continue blindly
- D. Iterate beyond `list.size()`

Q17. Java – Recursion Stack

A recursive function processes a linked list but causes `StackOverflowError` for very large lists. Which improvement is most appropriate when recursion is unnecessary?

- A. Convert the traversal to an iterative loop
- B. Add more recursive calls
- C. Increase the number of local variables
- D. Remove the termination condition

Q18. Java – HashMap Frequency

A frequency counter uses `get(x) + 1` without checking whether `x` exists. What is the safest concise correction?

- A. `map.put(x, map.getDefault(x, 0) + 1)`
- B. `map.put(x, map.get(x) - 1)`
- C. `map.clear()` before each insertion
- D. Use `containsKey(x)` and never update existing values

Q19. Java – Binary Search Overflow

A binary search computes mid as $(low + high) / 2$. With very large indexes, the sum may overflow. What is the safer expression?

- A. $low + (high - low) / 2$
- B. $(low * high) / 2$
- C. $high + low * 2$
- D. $low - high / 2$

Q20. Java – Two-Dimensional Arrays

A Java matrix may be jagged, meaning rows have different lengths. Which loop is correct?

- A. `j < matrix.length` for every row
- B. `j < matrix[i].length` for each row `i`
- C. `j <= matrix[i].length`
- D. `j < matrix[0].length` for all rows

Q21. Advanced – Sliding Window

A function finds the longest substring without repeating characters but moves the left pointer backward when a duplicate is found. What invariant should be maintained?

- A. The left pointer should never move backward; it should advance beyond the previous occurrence
- B. The window must always have even length
- C. The right pointer must reset to zero
- D. Every duplicate should be deleted from the string

Q22. Advanced – Monotonic Stack

A next-greater-element solution pops smaller values from a stack. What must be true about the stack after each iteration?

- A. It maintains a monotonic order suitable for future greater-element queries
- B. It contains every array element exactly once forever
- C. It must always be sorted in ascending order regardless of the problem
- D. It stores only negative values

Q23. Advanced – Dijkstra Stale Entries

A priority queue may contain multiple distances for one vertex. Which condition prevents processing stale entries?

- A. Skip when `poppedDistance != dist[node]`
- B. Reset `dist[node]` to zero
- C. Process only the largest distance
- D. Clear the queue after every edge

Q24. Advanced – Union Find

A Disjoint Set Union structure becomes slow because `find()` follows long parent chains. Which optimizations are appropriate?

- A. Path compression and union by rank/size
- B. Sorting all vertices after every union
- C. Rebuilding the graph after every find
- D. Removing parent pointers

Q25. Advanced – Topological Sort

A topological sort using Kahn's algorithm processes fewer than V vertices. What does this indicate?

- A. The graph contains a directed cycle or the input graph is not fully represented
- B. The graph must be complete
- C. Every vertex has indegree zero
- D. BFS has failed because queues cannot process DAGs

Q26. Advanced – Merge Sort

A merge function copies temporary values back using `arr[k] = temp[k]`. The merged segment starts at `l` rather than zero. What is the correction?

- A. Use `temp[k-l]` when writing back
- B. Use `temp[k+l]`
- C. Always write to `arr[0]`
- D. Copy only the first half

Q27. Advanced – DP Unreachable State

A minimum-cost DP uses `Integer.MAX_VALUE` as infinity and computes `dp[x] + cost`. What can go wrong?

- A. Integer overflow can occur; add cost only when `dp[x]` is reachable
- B. `MAX_VALUE` behaves exactly like zero
- C. The DP becomes greedy automatically
- D. The array must be sorted

Q28. Advanced – Memoization Key

A recursive solution memoizes results for a state described by `(index, remainingCapacity)`, but uses only `index` as the cache key. What is the consequence?

- A. Results for different capacities can overwrite or incorrectly reuse each other
- B. Memoization becomes more accurate
- C. The recursion becomes iterative

D. The capacity is automatically inferred

Q29. Advanced – Graph Recursion

A recursive DFS on a graph uses `visited[node] = true` only after exploring all neighbors. What is the principal risk?

- A. Cycles can cause repeated recursion or infinite recursion before the node is marked
- B. DFS becomes breadth-first
- C. The graph becomes disconnected
- D. The stack is automatically cleared

Q30. Advanced – Debugging Strategy

A corrected program passes the sample input but fails on hidden tests. Which debugging sequence is strongest?

- A. Test empty, minimum, maximum, duplicate, negative, boundary, and adversarial cases; then inspect invariants
- B. Increase the code length
- C. Remove assertions and tests
- D. Trust the sample because it passed

QUICK ANSWER KEY

1	B	2	B
3	A	4	B
5	B	6	A
7	A	8	A
9	A	10	B
11	A	12	A
13	A	14	B
15	A	16	A
17	A	18	A
19	A	20	B
21	A	22	A
23	A	24	A
25	A	26	A
27	A	28	A
29	A	30	A

Recommended practice: trace each program manually, write down the invariant, test boundary values, and then verify the corrected logic.